

Clautolisp User Manual

Codex

May 28, 2026

Contents

1	Scope	1
2	Synopsis	1
3	Modes of Operation	2
3.1	File mode	2
3.2	Expression mode	2
3.3	Interactive REPL	2
4	Options	3
4.1	Dialect selection	3
4.2	Input selection	4
4.3	REPL behaviour	4
4.4	Host backend	4
4.5	Colour output	4
4.6	Source-file and terminal encoding	5
4.7	Informational	6
5	Image-introspection variables (*AUTOLISP-...*)	6
5.1	Static — bound once, before the first user action	6
5.2	Dynamic — bound only while the matching action runs	7
6	Banner	7
7	Exit Codes	8
8	Errors and Diagnostics	8
9	Examples	9

10 Building the Executable	9
11 Versioning	10
12 clautolisp Extensions	10
12.1 Compositional list accessors (Cxxxr / Cxxxxr family)	10
12.2 ERROR	11
12.3 LCM, LOGXOR	11
12.4 VL-BT — full backtrace	11
12.5 VL-CATCH-ALL-ERROR-STACK — captured backtrace	11
12.6 The CLAL- namespace (clautolisp-only helpers)	12
12.7 Source	13
13 See Also	14

1 Scope

This manual documents the `clautolisp` standalone executable: how to invoke it, what options it accepts, and what it does in each mode.

It does **not** document the AutoLISP / Visual LISP language. For the language reference — surface syntax, builtins, semantics, and host-product divergences — see the companion `autolisp-spec` subproject (sibling directory `../../autolisp-spec/`).

This document must be kept in sync with the implementation. Whenever `clautolisp/tools/clautolisp/source/main.lisp` changes its option parsing, banner, exit codes, or supported modes, this file must be updated in the same commit.

The rendered PDF (`user-manual.pdf`, sibling of this file) is also checked in. Whenever this Org file changes, run `make user-manual` (or `make documentation`) to regenerate the PDF and stage both files in the same commit.

2 Synopsis

```
clautolisp [options] FILE.lsp
clautolisp [options] -x EXPRESSION
clautolisp [options]                               (interactive REPL)
```

The implementation is shipped as two image-built executables, one per supported Common Lisp host:

- `clautolisp-sbcl` — built on SBCL via `make build-clautolisp-sbcl`.
- `clautolisp-ccl` — built on CCL via `make build-clautolisp-ccl`.

Both binaries are produced under `clautolisp/tools/clautolisp/bin/` and accept the same command-line surface. The rest of this manual treats them interchangeably and refers to the binary as `clautolisp`.

3 Modes of Operation

`clautolisp` has three mutually exclusive modes; the mode is determined by the operands. In every mode the runtime installs the core builtin function bindings before evaluation begins.

3.1 File mode

A non-option argument is treated as a path to an AutoLISP source file. The reader loads the file under the active dialect, the evaluator processes every form sequentially, and the binary exits once the file has been fully consumed.

```
clautolisp drawing-tools.lsp
clautolisp --bricscad path/to/script.lsp
```

3.2 Expression mode

The `-x EXPRESSION` option evaluates the supplied string as if it were the entire source of a file, then exits.

```
clautolisp -x "(setq x 7) (+ x 3)"
clautolisp --bricscad -x "(princ (atof \"0x1p4\"))"
```

The expression is read with the source name `<-x>`, which appears in diagnostic messages.

3.3 Interactive REPL

With no `FILE` and no `-x`, `clautolisp` drops into an interactive read-eval-print loop on standard input.

- Primary prompt: `~_ $ ~`

- Continuation prompt (printed when an expression is incomplete): three spaces.
- Each balanced top-level form is evaluated and the result is printed using the AutoLISP value formatter (the same formatter used by `prin1`).
- A reader or runtime error is reported as a one-line diagnostic; the REPL then re-prompts.
- Closing standard input (Ctrl-D on a fresh line) ends the REPL.

The REPL accepts multi-line input: as long as the parser reports `:unexpected-eof`, the REPL keeps reading continuation lines and appends them to the in-progress form before retrying.

4 Options

All options begin with two dashes except for `-x`. They may appear in any order before, after, or interleaved with the `FILE` argument; the last occurrence of an option wins. `--` is **not** a recognised end-of-options marker; the runtime forwards every CLI argument to the parser.

4.1 Dialect selection

The dialect descriptor controls reader strictness, host-specific permissive parsing, and the runtime knobs documented in `autolisp-spec`. The default is the conservative `strict` profile.

`--dialect NAME` Select the named dialect. `NAME` is one of `strict`, `autocad-2026`, `bricscad-v26`.

`--strict` Equivalent to `--dialect strict`.

`--autocad` Equivalent to `--dialect autocad-2026`.

`--bricscad` Equivalent to `--dialect bricscad-v26`.

`--clautolisp` Equivalent to `--dialect clautolisp`.

The `clautolisp` dialect includes ‘clautolisp extensions’ (eg. variadic functions).

Whatever the dialect selected, all the operators are available. Only those that are not specifically inside the set defined by the dialect produce warnings when compiled and when used.

Unknown dialect names produce a diagnostic on standard error and exit code 1.

4.2 Input selection

-x EXPRESSION Evaluate EXPRESSION instead of reading a file. Implies expression mode. Must be followed by exactly one argument.

4.3 REPL behaviour

--quiet Suppress the REPL banner; only the prompt is shown. No effect outside REPL mode.

4.4 Host backend

The Host Abstraction Layer (HAL, Phase 8 of the implementation roadmap) decides where CAD-host effects land. The dialect descriptor controls the **language**; the host backend controls the **target**. Both axes are independent.

--host NAME Select the backend. Currently accepted: **mock** (default since 0.0.14): an in-memory deterministic CAD-database substitute capable of `entget` / `ssget` / `getvar` / `getstring` etc. **null**: every CAD-host operation signals **:host-not-supported** — useful for programs that should not touch a host at all. **live** is reserved for Phase 16.

--mock-input PATH Attach the file at PATH as the MockHost's prompt-stream. Lines are consumed in order by `getstring`, `getreal`, `getint`, `getpoint`, etc. so headless test rigs can feed deterministic answers to interactive scripts.

4.5 Colour output

`clautolisp` accents AutoLISP-symbol names in error messages, backtraces, and trace output with an ANSI colour when standard output is a terminal. The default is conservative (yellow on assumed-dark) and never fires when the output is being captured (pipe, file redirect, test harness).

--no-color Disable colour for this invocation. Equivalent to setting `$NO_COLOR` to any non-empty value (see <https://no-color.org>).

\$NO_COLOR Universal no-color convention; honoured equivalently to **--no-color**.

\$CLAUTOLISP_SYMBOL_FOREGROUND=NAME Force the foreground colour for printed AutoLISP symbols. Accepted names (case and whitespace insensitive): `black`, `red`, `green`, `yellow`, `blue`, `magenta`, `cyan`, `white` and their `bright-` prefixed variants (`bright-black` is also `grey` / `gray`). `default` / `none` / `off` leaves the foreground axis bare.

\$CLAUTOLISP_SYMBOL_BACKGROUND=NAME Force the background colour for printed AutoLISP symbols. Same vocabulary as the foreground sibling; either or both env vars may be set. Both win over the automatic luminance probe so the user gets a deterministic look without depending on terminal detection.

\$CLAUTOLISP_BACKGROUND=dark|light Override the terminal-background guess without specifying a colour. `dark` keeps the yellow accent; `light` flips it to blue.

\$CLAUTOLISP_PROBE_OSC11 Opt-in to the active OSC 11 terminal query (spawns `/bin/sh + stty + dd` against `/dev/tty`). Off by default because on some platforms — notably macOS arm64 — the helper subprocess receives SIGTTOU and hangs CLI startup. Safe to set on most Linux desktops and iTerm2 / Terminal.app launched from a normal shell.

Example: yellow symbols on a dim-blue background, deterministically, no probing:

```
export CLAUTOLISP_SYMBOL_FOREGROUND=bright-yellow
export CLAUTOLISP_SYMBOL_BACKGROUND=blue
clautolisp -x '(foreach x t (princ x))'
```

4.6 Source-file and terminal encoding

-e ENC Set the default source-file encoding for every `load` in the session, including nested (`load ...`) calls from init files. `ENC` is one of `utf-8`, `iso-8859-1`, `latin-1`, `windows-1252`, `cp1252`. Overrides the dialect default and the POSIX-locale guess (`LC_ALL` / `LC_CTYPE` / `LANG`). Surfaced to user code as the `*AUTOLISP-FILE-ENCODING*` global.

-E ENC Declared terminal-IO encoding for the session. Surfaced to user code as the ***AUTOLISP-TERMINAL-ENCODING*** global so code that emits raw bytes can adapt. No stream rebinding is performed at the CL level — set this when your downstream tooling needs the declared encoding to match what you actually emit. Same ENC vocabulary as **-e**.

4.7 Informational

--version Print `clautolisp` <version> on standard output and exit 0. The version is the value of `clautolisp.tools.clautolisp:*version*` (format `MAJOR.MINOR.DEVELOP`).

--help Print a short usage summary and exit 0.

5 Image-introspection variables (***AUTOLISP-...***)

At startup `clautolisp` publishes a set of globally-bound variables in the running AutoLISP image describing how it was invoked. User code can read them to branch on backend, dialect, encoding, etc. See `issues/closed/transmit-options.issue` for the full design and the alfe-side counterpart.

5.1 Static — bound once, before the first user action

AUTOLISP-BACKEND The symbol `CLAUTOLISP`.

AUTOLISP-DIALECT The active dialect’s canonical name as a symbol — `STRICT`, `AUTOCAD-2026`, `BRICSCAD-V26`.

AUTOLISP-HOST The HAL backend’s name as a symbol — `MOCK-HOST`, `NULL-HOST`, or `nil`.

AUTOLISP-MOCK-INPUT Pathname (string) passed to `--mock-input`, or `nil`.

AUTOLISP-GUI-COMMAND Subprocess renderer command passed to `--gui`, or `nil`.

AUTOLISP-TRACE `T` when `--trace` was on at startup.

AUTOLISP-FILE-ENCODING Value passed to `-e ENC` as a string, or `nil`.

AUTOLISP-TERMINAL-ENCODING Value passed to `-E ENC` as a string, or `nil`.

- *AUTOLISP-NO-INIT*** T when `-norc` / `--no-init` was passed, or the env-var override suppressed init files.
- *AUTOLISP-NO-COLOR*** T when `--no-color` was passed.
- *AUTOLISP-VERBOSE*** T when `-v` / `--verbose` was passed.
- *AUTOLISP-DEBUG*** T when `-d` / `--debug` was passed.
- *AUTOLISP-QUIET*** T when `-q` / `--quiet` was passed.
- *AUTOLISP-WILL-QUIT*** Reserved for the `--quit` flag; always nil under clautolisp today. alfe sets it.
- *AUTOLISP-MAIN*** Reserved for the `--main` flag; always nil under clautolisp today.
- *AUTOLISP-ACTIONS*** Ordered list of the actions parsed off the command line, each shaped `(load "PATH")`, `(eval "TEXT")`, or `(interactive T)`. Empty list when the implicit REPL is the only action.
- *AUTOLISP-HELP*** String containing the `--help` output.
- *AUTOLISP-VERSION*** String like "1.0.47".

User code may `setq` any of these; the runtime does not re-read the values, so the modification is purely an information change visible to subsequent `~boundp~/value` lookups.

5.2 Dynamic — bound only while the matching action runs

- *AUTOLISP-LOAD-PATHNAME*** The pathname of the file being loaded by a `-l` action. Bound to the path string for the duration of the load call; reset to nil on return (success or signal).
- *AUTOLISP-EXPRESSION*** The source text of a `-x` action, bound while the expression is evaluated. Cleared on return.
- *AUTOLISP-INTERACTIVE*** T while the REPL is active. Cleared on REPL exit.

Code running inside an init file can therefore distinguish "loaded because of `-l init.lsp`" (`*AUTOLISP-LOAD-PATHNAME*` points at `init.lsp`) from "loaded as an init file from the standard search path" (`*AUTOLISP-LOAD-PATHNAME*` is nil).

6 Banner

In REPL mode (and only in REPL mode) `clautolisp` prints a one-line banner before the first prompt:

```
clautolisp 0.0.9 - REPL (STRICT dialect, null-host host) - Ctrl-D to exit.
```

The version string, dialect name, and host backend name reflect the current build and the selected `--dialect` / `--host`. Pass `--quiet` to suppress the banner.

7 Exit Codes

`clautolisp` uses the following exit codes. They are stable and may be relied upon by shell scripts.

- 0 Successful completion (file or `-x` ran without error; REPL exited cleanly via end-of-input; `--help` / `--version` printed and exited; an AutoLISP `exit` / `quit` termination was raised cleanly).
- 1 A runtime error escaped the AutoLISP error-handling layer, or the option parser rejected an unknown flag.
- 2 The requested file could not be opened (caught Common-Lisp `file-error`).

Within the REPL, individual evaluation errors are reported but do **not** terminate the loop; the binary will still exit 0 once stdin closes.

8 Errors and Diagnostics

- Reader errors are produced by the AutoLISP reader and surface as `simple-error` conditions whose printable form embeds a structured `diagnostic` (severity, code, message, source span). They are printed to standard error in batch modes and to standard error in REPL mode while the loop continues.
- Runtime errors are signalled as `autolisp-runtime-error` with a symbolic `code` (e.g. `UNDEFINED-FUNCTION`, `UNBOUND-VARIABLE`, `TYPE-ERROR`) and a human-readable message.
- Termination conditions raised by `exit` / `quit` (`autolisp-termination`) surface as a one-line notice on standard error and exit cleanly.

The exact set of error codes lives in the runtime; see `autolisp-runtime/source/api.lisp` for the canonical mapping.

9 Examples

```
# Print a literal symbol.
$ clautolisp -x "(print 'hi)"
HI

# Evaluate a small program under the BricsCAD V26 dialect.
$ clautolisp --bricscad -x "(setq x 1) (+ x 2)"

# Run a script file under the AutoCAD 2026 dialect.
$ clautolisp --autocad some-script.lsp

# Interactive REPL, default dialect.
$ clautolisp
clautolisp 0.0.1 - REPL (STRICT dialect) - Ctrl-D to exit.
_$(setq xs (list 1 2 3))
(1 2 3)
_$(length xs)
3
_ $ ^D

# Interactive REPL with no banner.
$ clautolisp --quiet

# Show the version and exit.
$ clautolisp --version
clautolisp 0.0.1
```

10 Building the Executable

From the `clautolisp/` subproject root:

```
make build-clautolisp-sbcl # produces tools/clautolisp/bin/clautolisp-sbcl
make build-clautolisp-ccl  # produces tools/clautolisp/bin/clautolisp-ccl
```

Both targets reuse the `read-autolisp` build harness. The SBCL build sets `:save-runtime-options t` so the saved core forwards every command-

line argument to `clautolisp` rather than letting SBCL intercept `--help` or other runtime flags.

11 Versioning

The version string is stored in `clautolisp/tools/clautolisp/source/version.lisp` as `*version*`. The format is `MAJOR.MINOR.DEVELOP`. The `DEVELOP` counter is bumped on every change that touches `clautolisp` source code; the running binary's `--version` output is therefore a stable identifier of the build.

12 clautolisp Extensions

The core builtin registry installed by `clautolisp` is a **superset** of the operators listed in the local AutoLISP / Visual LISP specification draft. The extras documented in this section are convenience extensions and compatibility helpers that real-world AutoLISP code relies on without going through the spec process.

Code that targets the strict shared standard (the AutoCAD/BricsCAD intersection) **MUST NOT** depend on the operators below; tests written against `autolisp-test/harness/inventory.sexp` are intentionally restricted to the spec, and the inventory does not list these names.

12.1 Compositional list accessors (`Cxxxxr` / `Cxxxxxr` family)

The local specification documents `CAR`, `CDR` and `CADR`. Beyond those, `clautolisp` also installs:

two-level `CAAR`, `CDAR`, `CDDR`

three-level `CAAAAR`, `CAADR`, `CADAR`, `CADDR`, `CDAAR`, `CDADR`, `CDDAR`, `CDDDR`

four-level `CAAAAR`, `CAAADR`, `CAADAR`, `CAADDR`, `CADAAR`, `CADADR`, `CADDAR`, `CADDDR`,
`CDAAAR`, `CDAADR`, `CDADAR`, `CDADDR`, `CDDAAR`, `CDDADR`, `CDDDR`, `CDDDDR`

Each composition delegates to `CAR` / `CDR` so a non-list argument anywhere along the chain still surfaces a `CAR`-shaped or `CDR`-shaped runtime error under the proper operator name.

```
$ clautolisp -x "(caddr ' (a b c d e))"  
D
```

12.2 ERROR

(error MESSAGE-OR-STRING ...) raises an AutoLISP-visible runtime error with code :user-error and the supplied message. Modeled after the common Visual LISP idiom of calling (error ...) to abort with a user-facing message, and catchable with vl-catch-all-apply.

```
$ clautolisp -x "(vl-catch-all-error-message
                  (vl-catch-all-apply 'error '(\\"oops\\")))"
"oops"
```

12.3 LCM, LOGXOR

Convenience numerics that complement the in-spec GCD, LOGAND and LOGIOR. LCM takes two or more integers and returns their least common multiple; LOGXOR takes two or more integers and returns their bitwise exclusive-or.

```
$ clautolisp -x "(lcm 4 6)"
12
$ clautolisp -x "(logxor 12 10)"
6
```

12.4 VL-BT — full backtrace

The local specification reserves VL-BT as a Visual LISP backtrace operator. clautolisp implements it as a real backtrace printer rather than a stub: (vl-bt) prints the AutoLISP call stack at the current evaluation point, one line per frame, most recent first.

The runtime tracks the stack through *autolisp-call-stack*, a dynamically-bound list of evaluator frames pushed by autolisp-eval and by every SUBR / USUBR call. Frame kinds are :eval, :special-op, :subr and :usubr.

12.5 VL-CATCH-ALL-ERROR-STACK — captured backtrace

(vl-catch-all-error-stack OBJECT) is a clautolisp-only extension companion to the standard vl-catch-all-error-message. Given a catch-all error object, it returns the AutoLISP call-stack snapshot captured at the time the underlying autolisp-runtime-error was raised. Each frame is a cons (KIND . PAYLOAD), where PAYLOAD is either the form being evaluated (for :eval / :special-op) or the cons (NAME . ARGS) of the function call (for :subr / :usubr).

This is the operator the `autolisp-test` harness calls to render an AutoLISP backtrace inside a FAIL detail when a tested function raises. AutoCAD and BricsCAD do not expose the captured stack, so the harness gracefully falls back to message-only reporting on those hosts.

```
$ clautolisp -x "(setq c (vl-catch-all-apply 'car '(7)))
                  (length (vl-catch-all-error-stack c))"
;; one or more frames captured at the time CAR was rejected
3
```

Code that needs to remain portable should test for the operator before calling it:

```
(if (boundp 'vl-catch-all-error-stack)
    (vl-catch-all-error-stack object)
    nil)
```

12.6 The CLAL- namespace (clautolisp-only helpers)

The four other sub-sections above (`Cxxxxr` accessors, `ERROR`, `LCM~/~LOGXOR`, `VL-BT~/~VL-CATCH-ALL-ERROR-STACK`) cover extensions that complete an established AutoCAD/BricsCAD idiom without inventing a new one. From clautolisp 1.0.76 onward, a second class of extensions ships under a reserved `CLAL-` prefix: helpers that have **no** vendor counterpart and that we want to make trivially distinguishable from any future Autodesk- or Bricsys- documented name. Code that calls one of these functions is explicitly opting into the clautolisp dialect.

The probing convention for portable code:

```
(if (boundp 'clal-sysvar-list)
    (clal-sysvar-list)
    ;; vendor fallback -- there isn't one; either accept that the
    ;; query is unavailable or fall back to the SETVAR command.
    nil)
```

12.6.1 CLAL-SYSVAR-LIST – list every sysvar known to the host

`(clal-sysvar-list)` returns a list of strings: the sysvar names the active host (mock or live) reports, sorted in case-sensitive ASCII order. On a default `MockHost` the list has 1836 entries – the same universe as the autolisp-spec §16 **System Variables** catalogue.

```
$ clautolisp -x "(length (clal-sysvar-list))"
1836
```

```
$ clautolisp -x "(car (clal-sysvar-list))"
"3DCONVERSIONMODE"
```

12.6.2 CLAL-SYSVAR-APROPOS – substring search over sysvar names

(clal-sysvar-apropos PATTERN) returns the subset of (clal-sysvar-list) whose names contain PATTERN as a case-insensitive substring. Useful at the REPL when you remember “there’s something with ANG in it” but not the exact name.

```
$ clautolisp -x "(clal-sysvar-apropos \"ANG\")"
("ANGBASE" "ANGDIR" "INPUTHISTORYMODE" ...)
```

```
$ clautolisp -x "(clal-sysvar-apropos \"ang\")"      ;; case-insensitive
("ANGBASE" "ANGDIR" "INPUTHISTORYMODE" ...)
```

```
$ clautolisp -x "(clal-sysvar-apropos \"ERRNO\")"
("ERRNO")
```

```
$ clautolisp -x "(length (clal-sysvar-apropos \"\"))"  ;; empty -> all
1836
```

A non-string PATTERN raises :invalid-argument. A pattern that matches nothing returns nil.

The full normative entries for both functions live in `autolisp-spec §16 clautolisp Extensions`; this section is a quick-reference summary.

12.7 Source

The implementation of every operator listed above lives in:

- `clautolisp/autolisp-builtins-core/source/api.lisp` — registration and per-builtin code.
- `clautolisp/autolisp-builtins-core/README.org` — module-level rationale, kept in sync with this manual section.
- `clautolisp/autolisp-runtime/source/api.lisp` — backtrace tracking (`*autolisp-call-stack*`, `current-autolisp-call-stack`, the

call-stack slot on `autolisp-runtime-error`, and the `autolisp-catch-all-error-call-stack` accessor).

13 See Also

- `../../autolisp-spec/` — AutoLISP / Visual LISP language reference, including dialect divergences and the normative source for builtin semantics.
- `PLAN.md` — Implementation roadmap and per-phase completion notes.
- `clautolisp/autolisp-reader/` — Reader subsystem (lexer, parser, diagnostics, dialect descriptors).
- `clautolisp/autolisp-runtime/` — Runtime object model and evaluator.
- `clautolisp/autolisp-builtins-core/` — Core builtin function registry (the layer that supplies `print`, `+`, `atof`, etc. consumed by the CLI).
- `../../autolisp-test/` — Conformance test suite. Uses `vl-catch-all-error-stack` and `vl-bt` to render AutoLISP backtraces inside FAIL details when a tested function raises on clautolisp.